Analysis Report for HW2_Phacha

https://github.com/Tasee-P/NLP_HW2_Phacha

1. Work Process and Summary output

I structured my workflow to leverage the provided class materials and homework templates, which helped minimize errors. A summary of my execution is below:

- Task 1 (Training Word Embedding Models): I used the exact load_dataset snippet from Hugging Face (dataset = load_dataset("wikimedia/wikipedia", "20231101.simple", split="train")) provided in the homework instructions. After applying text normalization, I successfully trained and saved both a Continuous Bag-of-Words (CBOW) and a Skip-gram model using the gensim library.

- Task 2 (Comparing Word Embeddings) I adapted code from the NLP_W26_embeddings.ipynb class notebook and compared my models against two large legacy models from the gensim.downloader API: word2vec-google-news-300 and glove-wiki-gigaword-100. I tested them using five semantic queries, including analogies and similarity tests. Both models used 100-dimensional vectors with a context window of 5, but differ in training objective via the sg hyperparameter: CBOW (sg=0) predicts a target word from the surrounding context, while Skip-gram (sg=1) predicts surrounding words from a target.

  Outputs: Analogy (Rome - Italy + Japan): Google News returned 'japanese' and 'tokyo'. Custom models returned 'han' and 'emperors' || Similarity (Tea): Google News returned 'activist_Jamie_Radtke' and 'Tea_NASDAQ_PEET'. Custom models returned 'coffee', 'pudding', and 'soup'.

- Task 3 (WEFE Bias): I utilized the WEFE library as instructed in the prompt. To resolve a formatting bug and ensure the tests ran correctly, I applied the specific bug-fix snippet: metric_params={'preprocessors': [{}, {}, {'lowercase': True}]}. I designed an original WEAT query to test the subconscious bias between Age (Youth vs. Elderly) and Technology vs. Nature/Traditional concepts.

  - Target Sets: Youth (child, teen, young) vs. Elderly (old, senior, grandparent)
  - Attribute Sets: Technology (computer, internet, digital) vs. Nature/Traditional (garden, traditional, nature)

  Outputs: WEFE Scores: Custom CBOW (0.66), Custom Skip-gram (0.54), Google News (0.16), and GloVe (0.46).

- Task 4 (Classification): I implemented the CountVectorizer and LogisticRegression setup from the NLP_W26_LogReg.ipynb class notebook. I adapted the class setup by replacing the IMDB dataset with the Rotten Tomatoes dataset. After briefly experimenting with other datasets (like PetFinder) and obtaining similar results, I successfully compared a 10,000-dimension Sparse model against my custom 100-dimension Dense model.

  Outputs: Accuracies: Sparse (77.02%) and Dense (62.26%).

2. Analysis

1. Embeddings

Training on a Data Ocean (The Role of Corpus Size): I evaluated the models using five queries (three vector analogies and two pure similarities). The results highlighted the limitations of training on a restricted corpus like Simple Wikipedia compared to massive legacy models:

Analysis of the Results

- **Analogies and Corpus Brittleness:** The massive legacy models successfully handled multi-step vector math, accurately returning 'sister' (Brother - Man + Woman), 'ran' (Swam - Swim + Run), and 'tokyo' (Rome - Italy + Japan). In contrast, my custom models failed across other analogy types, returning 'wife', 'pepsicola', and 'han' respectively. This proves that a massive, diverse training corpus is strictly required to build generalized, robust geometric vector spaces.
- **Word Sense Disambiguation:** The geographic analogy (Rome - Italy + Japan) exposed a critical limitation in the smaller corpus: word sense disambiguation. While the legacy models correctly associated "Rome" with a modern capital city (yielding 'tokyo' or 'japanese'), my custom models returned ancient historical concepts ('han', 'byzantine, 'emperors'). Because the Simple Wikipedia corpus heavily discusses Rome in the context of the Ancient Roman Empire rather than the modern terms.
- **Similarity Specificity vs. Noise:** For pure similarities, corpus size drastically changed the focus. For 'guitar', custom models returned broad musical concepts (bass, drums), while legacy models returned highly specific variations (acoustic_guitar, mandolin). However, massive datasets also introduce distinct noise. For the word 'tea', the custom models returned basic, related categories (coffee, soup, pudding). The Google News model, returned highly specific but irrelevant real-world entities like 'activist_Jamie_Radtke' and the stock ticker 'Tea_NASDAQ_PEET'. (This issue occurred in a recent run; however, this part is generally stable across most runs., additional: "PEET" probably refers to Peet's Coffee, but it is only loosely relevant.)

2. Bias

Consequences of Biased Embeddings

Analysis of the Results

The results demonstrated a clear disparity: the custom-trained Simple Wikipedia models exhibited significantly higher bias scores (CBOW: 0.66, Skip-gram: 0.54) compared to the massive legacy models (Google News: 0.16, GloVe: 0.46). A higher positive score mathematically indicates a stronger stereotypical association of younger age with technology terms. This suggests that the smaller, restricted corpus of Simple Wikipedia may contain highly concentrated, stereotypical language regarding age. In contrast, while the massive datasets behind Google News and GloVe still exhibit measurable bias, their vast diversity of contexts likely dilutes this specific stereotype, resulting in lower overall association scores.

If these embeddings are used as feature inputs for downstream machine learning models, the consequences could result in direct allocative harm. For example, if an automated resume screening system uses these word vectors to rank candidates for a tech-heavy role, the model could mathematically penalize older applicants. Because the underlying vector space lacks semantic proximity between "Elderly" terms and "Technology" terms, the AI might incorrectly score older candidates as less relevant to the job. This example is only illustrative; the analysis in this work does not specifically test such an application.

Reflections on the Framework's Limitations

- **The Limitations of Binary Bias Testing:** I initially hypothesized (just roughly thought in my mind) that adding modern terms like "AI" to the Technology set might push the association closer to the "Elderly" target group, driven by labor economics where automation leads to labor replacement among established adult workers. I considered adding "unemployment" to capture this but realized a fundamental technical limitation: WEAT evaluates bias using direct cosine similarity between flat, rigid categories. "Unemployment" is not a direct synonym for "technology," but a downstream socio-economic effect. Standard WEAT tests cannot model multi-step, causal relationships (e.g., Technology causes Unemployment, which impacts Older Adults), meaning these binary evaluation metrics often oversimplify complex real-world dynamics.
- **Researcher Bias and Vocabulary Selection:** My specific choice of vocabulary inherently influenced the test outcome. For the "elderly" target set, relying on terms like "grandpa" places the target in domestic contexts rather than professional ones. Utilizing economic terms like "veteran worker" or "retiree" might have yielded a fundamentally different cosine similarity to "technology." Because these embeddings exist in a highly complex, 100-dimensional space, human intuition is a poor predictor of mathematical proximity. Assuming a specific list of words is the "correct" baseline is a form of researcher bias, highlighting the need for testing diverse synonym sets to avoid hardcoding semantic blind spots into evaluation metrics. However, in selecting the target words, I attempted to reduce potential bias by using general and commonly recognized

terms such as “grandpa.” (While I tried to reduce my own bias, I may have selected inappropriate or biased group words.)

3. Classification

To evaluate the downstream impact of these embeddings, I conducted a binary sentiment classification experiment using the Rotten Tomatoes movie review dataset.

- I first trained a Sparse Model utilizing a CountVectorizer restricted to a maximum of 10,000 features, which achieved an accuracy of 77.02%.
- I then trained a Dense Model utilizing the custom-trained CBOW model to convert reviews into averaged 100-dimensional vectors, which achieved an accuracy of 62.26%.

Analysis of the Results

The Sparse model significantly outperformed the Dense model, despite the Dense model theoretically capturing deeper semantic meaning. The reason the Sparse model (10,000 features) performed much better than the Dense model (100 features) comes down to the math used in Task 4: return np.mean(vectors, axis=0).

For the Dense model, we take the mathematical vector for every single word in a movie review and average them together into a single 100-dimensional vector representation for the entire review.

The Sparse model doesn't average anything. It treats every single word as its own separate checklist item, preserving their independent presence and weight. Because the words stay completely separate, the computer can easily spot the exact positive or negative words without them washing each other out.

3. Reflection

✓ **I appreciated the clear instructions and starter code provided** in the homework PDF, as well as the permission to use the code demonstrated in class which made the initial setup smooth and saved time. I encountered a few technical problems but was able to fix them quickly.

✓ **Training Data Matters:** Based on the results of Wikipedia vs. Google News. The results highlights that while a small corpus can learn basic similarity, accurately mapping geometric analogy vectors requires a massive, diverse training corpus. Could this be why experts predict Gemini will ultimately dominate the AI landscape, given its access to Google’s data ocean as a foundation?

✓ **Model Comparison (Sparse vs. Dense):** The Sparse model significantly outperformed the Dense model in terms of accuracy, though it required 100 times the number of features

(10,000 vs. 100). The Dense model was vastly more computationally efficient but suffered a notable drop in predictive performance. I struggled with this part initially due to a "silent crash" (the cell stopped running without throwing an error), and I didn't realize the dense output was missing until I analyzed the results.

✓ **Code Adaptation and Active Learning:** While I utilized the provided class notebooks as a foundational template, I ensured active engagement with the material by adapting and thoroughly commenting the code in my own words. This line-by-line analysis deepened my understanding of critical machine learning mechanics. For instance, it clarified the fundamental difference between feature dimensionality (restricting the data breadth to 10,000 vocabulary words via max_features) and model convergence parameters (allocating 1,000 training iterations via max_iter to ensure the model successfully finds an answer. Additionally, exploring the WEFE framework taught me the exact mechanics of bias evaluation. I learned that the Word Embedding Association Test (WEAT) calculates scores based strictly on the positional order of the inputs—specifically, how Target 1 relates to Attribute 1. Understanding this underlying mathematical structure is essential; knowing exactly how the query is built is what makes it possible to accurately interpret whether a positive or negative result indicates a specific bias.

✓ **Future Improvements and Goal Setting:** If I were to redo this part, I would establish a more specific, application-driven goal or unemployment-technology goal for the bias evaluation before constructing my query sets. For instance, rather than testing a broad demographic concept like Age versus Technology, I would investigate socioeconomic associations by comparing terms related to 'Employment' versus 'Unemployment' against 'Technology'. Setting a highly focused objective like this would allow me to analyze if these models inherently associate AI and technological advancement with job loss or creation. This would yield actionable insights that directly apply to the realities of the modern AI technology job market. (

✓ **Data Partitioning and Workflow Abstraction:** Reflecting on traditional machine learning pipelines, I noted the absence of a manual data splitting step (such as a standard 80/20 train-test split). I later realized that Python and the Hugging Face library abstract this step away behind the scenes. While this saves time and ensures my evaluation is exactly consistent with other researchers using the same dataset, it is a good reminder of how modern NLP libraries hide core ML mechanics. Final thought on this, Without a strong grasp of the underlying fundamentals, executing a few lines of code can feel like magic, creating a false sense of capability. However, it is the understanding of these "hidden" mechanics—the mathematical trade-offs, the data partitioning, and the structural limitations—that creates a definitive boundary between those who simply use AI as a black box and those who truly understand how to engineer and evaluate it.

✓ **Iterative Testing and Anomaly Verification:** During the final review of the semantic queries, I observed an unexpected result: my custom models initially performed well on specific geographic vector analogies. This was surprising because the relatively small Simple Wikipedia dataset appeared nearly as effective as much larger legacy models,

suggesting that dataset size might not substantially impact analogy performance. To investigate, I modified the query vocabulary and conducted a second round of testing. This iterative evaluation revealed the true pattern: the custom models failed on the new geographic analogies, while the legacy models improve in geography performance. Some noise was also noted in the latest similarity queries (for legacy models). Overall, the results reaffirm the expected hypothesis that larger datasets generate more robust and reliable semantic spaces. This experience underscores a key lesson: initial outputs can be misleading, and rigorous re-testing with varied inputs is essential to accurately assess model capabilities.

✓ **I noticed that the more I re-ran the Skip-gram model for Task 3, the lower the bias numbers became.** While I haven't fully investigated the exact mathematical cause yet, this is likely because Word2Vec models rely on random initialization and negative sampling during training; without setting a fixed random seed, the model's weights and vector associations will naturally fluctuate every time it compiles ((even when restarting the kernel before re-running). I also noticed that the legacy Google News and GloVe models remained perfectly steady, while my custom CBOW model fluctuated only within a very small range. This discrepancy highlights the difference between trained models. The legacy models are loaded from pre-trained files, in contrast, my custom models are trained from scratch and rely on random initialization, that could be why its vector fluctuate every time the code compiles.

✓ **During the coding process, I was uncertain whether there were any mistakes in my implementation.** For instance, in the youth group I included both "child" and "children" as separate terms. After completing the analysis, I realized that treating them as different words might not have been the best choice. I also questioned whether the number of words in each group affects the results and whether each group should contain an equal number of terms. My assumption is that the exact number may not significantly affect the outcome. I am not entirely certain.

✓ **Unfortunately, I experienced some AI glitches and hallucinations.** Because of that, I realized that critical thinking is very important. I noticed that the AI can be very confident even when it is wrong, and it can also get confused when I keep testing it. I did not rely on them, I like to play test sometimes. And I believe the more I learn about it, the better I can handle these issues. I am not trying to be smarter than AI, but fully relying on it can be risky.

4. AI Clarification & Usage

During the development and execution of the codebase, an AI assistant was utilized to brainstorm ideas and troubleshoot technical roadblocks:

- Execution Order: I encountered a problem where !pip installations were throwing errors. The AI suggested that I should not put everything in the same block, explaining that running code out of sequence before installations finish can cause errors.

- Google Colab "Silent Crash" (Out of Memory): While generating the averaged vectors for the Dense model, the notebook experienced a silent crash where execution halted without throwing an error.
- NumPy Version Conflict: A dependency conflict with a newer version of NumPy (2.0) caused compatibility issues with the WEFE framework. The assistant suggested force-installing a stable older version (numpy==1.26.4) to restore full functionality. However, I did not follow that suggestion. Sometimes the !pip output appeared normal, and other times it showed warnings (in red), but the installation still completed successfully.

5. Annex: Screenshots of the Output

Note: The output generated in the Colab notebook may vary slightly across runs.

First: Task 2

```
*****The legacy model*****
Running 5 semantic queries

-- Analogy: King - Man + Woman
My CBOW            : ['queen', 'throne', 'vi']
My Skip-gram       : ['queen', 'viii', 'vii']
My Google News (Legacy): ['queen', 'monarch', 'princess']
My GloVe (Legacy)   : ['queen', 'monarch', 'throne']

-- Analogy: Paris - France + Germany
My CBOW            : ['berlin', 'munich', 'frankfurt']
My Skip-gram       : ['berlin', 'munich', 'leipzig']
My Google News (Legacy): ['berlin', 'german', 'lindsay_lohan']
My GloVe (Legacy)   : ['berlin', 'frankfurt', 'vienna']

-- Analogy: Walked - Walk + Run
My CBOW            : ['passed', 'drafted', 'managed']
My Skip-gram       : ['chased', 'dragged', 'parole']
My Google News (Legacy): ['ran', 'drove', 'runs']
My GloVe (Legacy)   : ['went', 'drove', 'ran']

-- Similarity: 10 closest to 'piano'
My CBOW            : ['violin', 'concerto', 'cello', 'harpsichord', 'quartet', 'flute', 'instrument', 'clarinet', 'orchestra', 'concertos']
My Skip-gram       : ['violin', 'cello', 'harpsichord', 'concerto', 'quartet', 'concertos', 'clarinet', 'flute', 'orchestral', 'quartets']
My Google News (Legacy): ['violin', 'cello', 'clarinet', 'saxophone', 'pianist', 'trombone', 'harpsichord', 'pianos', 'guitar', 'classical_guitar']
My GloVe (Legacy)   : ['violin', 'cello', 'guitar', 'flute', 'concerto', 'saxophone', 'clarinet', 'orchestral', 'harpsichord', 'compositions']

-- Similarity: 10 closest to 'coffee'
My CBOW            : ['chocolate', 'beans', 'sugar', 'cotton', 'wine', 'fiber', 'starch', 'vegetable', 'dairy', 'corn']
My Skip-gram       : ['chocolate', 'vegetables', 'drinks', 'cider', 'beans', 'potatoes', 'cocoa', 'drink', 'starbucks', 'roasted']
My Google News (Legacy): ['coffees', 'gourmet_coffee', 'Coffee', 'o_joe', 'Starbucks_coffee', 'coffee_beans', 'latté', 'cappuccino', 'brewed_coffee', 'espresso']
My GloVe (Legacy)   : ['tea', 'drinks', 'beer', 'cocoa', 'wine', 'drink', 'corn', 'sugar', 'bread', 'fruit']
```

Lastest: Task 2

```
*****The legacy model*****
[==================================================] 100.0% 1662.8/1662.8MB downloaded
[==================================================] 100.0% 128.1/128.1MB downloaded
Running 5 semantic queries

-- Analogy: Brother - Man + Woman
My CBOW            : ['wife', 'husband', 'cousin']
My Skip-gram       : ['cousin', 'daughter', 'husband']
My Google News (Legacy): ['sister', 'daughter', 'mother']
My GloVe (Legacy)   : ['daughter', 'wife', 'mother']

-- Analogy: Rome - Italy + Japan
My CBOW            : ['han', 'jerusalem', 'romans']
My Skip-gram       : ['emperors', 'byzantine', 'emperor']
My Google News (Legacy): ['japanese', 'korean', 'tokyo']
My GloVe (Legacy)   : ['tokyo', 'beijing', 'seoul']

-- Analogy: Swam - Swim + Run
My CBOW            : ['hearne', 'dink', 'supervisor']
My Skip-gram       : ['pepsicola', 'booths', 'compassionate']
My Google News (Legacy): ['ran', 'runs', 'drove']
My GloVe (Legacy)   : ['ran', 'went', 'drove']

-- Similarity: 10 closest to 'guitar'
My CBOW            : ['bass', 'drums', 'percussion', 'vocals', 'rhythm', 'keyboard', 'guitars', 'saxophone', 'harmonica', 'trumpet']
My Skip-gram       : ['drums', 'bass', 'vocals', 'percussion', 'guitars', 'backing', 'acoustic', 'instruments', 'harmonica', 'keyboards']
My Google News (Legacy): ['guitars', 'acoustic_guitar', 'electric_guitar', 'mandolin', 'harmonica', 'saxophone', 'bass_guitar', 'acoustic_guitars', 'sax', 'banjo']
My GloVe (Legacy)   : ['bass', 'guitars', 'vocals', 'drums', 'harmonica', 'percussion', 'piano', 'acoustic', 'saxophone', 'keyboards']

-- Similarity: 10 closest to 'tea'
My CBOW            : ['cooking', 'coffee', 'wine', 'soup', 'leather', 'chocolate', 'pudding', 'potato', 'flour', 'boots']
My Skip-gram       : ['coffee', 'matcha', 'pudding', 'tofu', 'mint', 'chocolate', 'noodle', 'banana', 'soup', 'lemonade']
My Google News (Legacy): ['Tea', 'teas', 'shape_Angius', 'activist_Jamie_Radtke', 'decaffeinated_brew', 'planter_bungalow', 'herbal_tea', 'coffee', 'jasmine_tea', 'Tea_NASDAQ_PEET']
My GloVe (Legacy)   : ['coffee', 'beer', 'drinks', 'drink', 'lunch', 'fruit', 'wine', 'teas', 'cocktail', 'juice']
```

First: Task 3

```
... Running custom WEAT Extension: Age vs. Technology Bias

    WEAT Bias Results (Positive score = Youth/Tech bias, Negative = Elderly/Tech bias):
    model_name                                  My CBOW  My Skip-gram  \
    query_name
    Youth and Elderly wrt Technology and Nature     0.73          0.41

    model_name                                  MY Google News  My GloVe
    query_name
    Youth and Elderly wrt Technology and Nature            0.16      0.46
```

Second: Task 3

```
... Running custom WEAT Extension: Age vs. Technology Bias

    WEAT Bias Results (Positive score = Youth/Tech bias, Negative = Elderly/Tech bias):
    model_name                                  My CBOW  My Skip-gram  \
    query_name
    Youth and Elderly wrt Technology and Nature     0.78          0.29

    model_name                                  MY Google News  My GloVe
    query_name
    Youth and Elderly wrt Technology and Nature            0.16      0.46
```

Lastest: Task 3

```
... Running custom WEAT Extension: Age vs. Technology Bias

    WEAT Bias Results (Positive score = Youth/Tech bias, Negative = Elderly/Tech bias):
    model_name                                  My CBOW  My Skip-gram  \
    query_name
    Youth and Elderly wrt Technology and Nature     0.66          0.54

    model_name                                  MY Google News  My GloVe
    query_name
    Youth and Elderly wrt Technology and Nature            0.16      0.46
```

First: Task 4

```
--- Task 4: Sample Review and Label ---
Review: the rock is destined to be the 21st century's new " conan " and that he's going to make a splash even greater than arnold schwarzenegger , jean-claud van damme or steven segal .
Label: 1
--------------------------------------

Training Model SPARSE
Sparse Model Accuracy: 0.7702
Sparse Feature Dimensionality: 10000 features

Training Model Dense
Dense Model Accuracy: 0.6220
Dense Feature Dimensionality: 100 features
```

Latest: Task 4

```
Loading dataset for Sentiment Analysis
--- Sample Review and Label ---
Review: the rock is destined to be the 21st century's new " conan " and that he's going to make a splash even greater than arnold schwarzenegger , jean-claud van damme or steven segal .
Label: 1

Training Model SPARSE
Sparse Model Accuracy: 0.7702
Sparse Feature Dimensionality: 10000 features

Training Model Dense
Dense Model Accuracy: 0.6266
Dense Feature Dimensionality: 100 features
```